TABLE DES MATIÈRES

Page	Thème
1	Fragmentation (FH, FV, Mixte, Dérivée)
2	Requêtes distribuées & réécriture SQL
3	BNLJ & Optimisation coût
4	Algorithmes de jointure avancés
5	Index (B-Tree, Bitmap, BJI)
6	OLAP & Schéma étoile
7	Vues Matérialisées & SQL avancé
8	Exercice : Fragmentation & Requêtes réparties
9	Exercice : Coût & Jointures
10	Exercice : Index, RBA & VM

RAPPELS CRITIQUES

BNLJ : Coût = B(R) + ⌈B(R)/M⌉ × B(S)

Hash Join : 3(|R|+|S|) blocs ; M ≥ √B(R) + 1

VM CONCURRENTLY → index unique *obligatoire*

⚠ **Piège :** VH : si colonne du prédicat H a été projetée hors de V → fragment impossible

⚠ **Piège :** COUNT(*) ≠ COUNT(DISTINCT x) quand JOIN peut dupliquer des tuples

ARBRE : QUELLE JOINTURE CHOISIR ?

```
Quelle jointure choisir ?
  Données triées sur attribut join  → Sort-Merge Join
  Non triées, grandes tables        → Hash Join  (p.4)
  Index disponible sur table interne → Index NLJ
  M très petit / tables petites     → BNLJ       (p.3)
  Réduction transfert réseau (dist.) → Semi-jointure (p.2)
```

ARBRE : QUEL INDEX CHOISIR ?

```
Quel index ?
  Haute cardinalité + range/ORDER BY → B-Tree  (p.5)
  Faible cardinalité + OLAP/AND/OR   → Bitmap  (p.5)
  Multi-colonnes fréquentes ensemble  → B-Tree composite
  Jointure étoile, filtre multi-dim.  → BJI     (p.5)
  Agrégat coûteux répété              → VM      (p.7)
```

ARBRE : TYPE DE FRAGMENTATION ?

```
Fragmentation ?
  Lignes selon prédicat (WHERE)       → FH  σ_p(R)
  Colonnes selon usage (SELECT cols)  → FV  π_A(R)
  Lignes PUIS colonnes                → Mixte HV
  Colonnes PUIS lignes                → Mixte VH ⚠ risque
  Table faits suit dimension          → Dérivée (p.1)
```

ARBRE : VM OU INDEX DIRECT ?

```
Index ou VM ?
  Agrégat répété > 100k lignes        → VM
  Faible cardinalité OLAP             → Bitmap / BJI
  Range + ORDER BY fréquent           → B-Tree composite
  Multi-filtres schéma étoile         → BJI + FHD
```

FORMULES & RÈGLES À CONNAÎTRE

Reconstructibilité FH : ∪ σ_pi(R) = R (prédicats couvrent R, disjoints)

Reconstructibilité FV : F1 ⋈_PK F2 ⋈_PK … = R (PK dans chaque fragment)

Mémoire min. BNLJ : M_min = √B(R) + 1 (coût = 3·B(R) atteint)

Réduction semi-jointure : SJ(R,S,A) = R ⋈ π_A(S)

Index composite (préfixe) : idx(A,B,C) → utilisable SSI A dans prédicat

FRAGMENTATION HORIZONTALE (FH)

Opérateur : `Ri = σ_pi(R)` — sélection par prédicat

Reconstruction : `R = R1 ∪ R2 ∪ … ∪ Rn`

Propriétés obligatoires :

- **Complétude :** ∀t ∈ R, ∃i : t ∈ Ri
- **Disjonction :** Ri ∩ Rj = ∅ (prédicats mutuellement exclusifs)
- **Reconstruction :** ∪ Ri = R

Ex. `R1 = σ_{Pays='FR'}(R)`, `R2 = σ_{Pays='DE'}(R)`

FRAGMENTATION VERTICALE (FV)

Opérateur : `Ri = π_Ai(R)` — projection sur colonnes

Reconstruction : `F1 ⋈_PK F2 ⋈_PK …` (PK dans chaque fragment)

⚠ **Piège :** La clé primaire DOIT être dupliquée dans chaque fragment vertical

Ex. `F1 = π_{id,nom,adresse}(R)`, `F2 = π_{id,salaire}(R)` → jointure sur `id`

FRAGMENTATION MIXTE HV & VH

	HV (H puis V)	VH (V puis H)
Ordre	σ → π	π → σ
Reconstr.	∪ puis ⋈_PK	⋈_PK puis ∪
Nb frags	nH × nV	≤ nV × nH

⚠ **Piège :** VH : si la colonne du prédicat de H a été projetée hors de V, le fragment H devient vide ou invalide

Ex. HV : `F11 = π_{id,salaire}(σ_{dept='RH'}(R))` — lignes RH, col salaire uniquement

FRAGMENTATION DÉRIVÉE

Fragments obtenus par jointure avec fragments d'une autre table : `Ri = π_cols(R ⋈ Si)`

Usage : distribuer une table en suivant la localisation d'une autre (co-localisation).

Ex. Si `Commandes` est fragmentée par pays, fragmenter `LignesCommande` de la même façon via `LignesCommande ⋈ Commandes_i`

TABLEAU RÉCAP RECONSTRUCTION

Type	Opérateur	Condition
FH	∪	Prédicats couvrent R
FV	⋈_PK	PK dans chaque frag.
Mixte HV	∪ puis ⋈	Reconstituer H, puis V
Mixte VH	⋈ puis ∪	Joindre V, puis réunir H

EXERCICE TYPE — IDENTIFIER & RECONSTRUIRE

Énoncé : Table `R(id, dept, salaire, ville)` découpée en :
`F1 = π_{id,dept}(σ_{ville='Paris'}(R))`
`F2 = π_{id,dept}(σ_{ville='Lyon'}(R))`
`F3 = π_{id,salaire}(R)`

1. Quel type ?

Ex. F1, F2 : FH sur ville. F3 : FV sur colonnes. Ensemble : Mixte HV (F1∪F2 = σ horizontal, puis F3 projection = vertical)

2. Vérifier complétude :

Ex. Prédicats `ville='Paris'` et `ville='Lyon'` — couvrent-ils tous les tuples ? Non si ∃ ville='Marseille' → fragmentation incomplète !

3. Reconstruction :

Ex. Étape 1 : `Fhoriz = F1 ∪ F2` (colonnes id, dept)
Étape 2 : `R = Fhoriz ⋈_{id} F3`

⚠ **Piège :** Si id absent de F3, la jointure est impossible → FV invalide

PRINCIPES DE RÉÉCRITURE DISTRIBUÉE

Principe : remplacer `SELECT … FROM R` par une UNION de requêtes sur les fragments `Ri`

Algorithme :

1. Décomposer le `WHERE` en prédicats
2. Pour chaque fragment, tester si son prédicat *contredit* la condition
3. Éliminer les fragments contradictoires (résultat vide garanti)
4. Combiner les fragments restants avec `UNION`

Push-down : appliquer σ et π *avant* les jointures inter-sites → réduction des transferts réseau

COÛT DE TRANSFERT RÉSEAU

Modèle de coût distribué :
Coût total = Coût I/O local + Coût transfert réseau
`Coût_réseau = nb_tuples_transférés × taille_tuple × coût_par_octet`

Minimiser les transferts :
1. Push-down σ : filtrer avant d'envoyer
2. Push-down π : ne projeter que les colonnes nécessaires
3. Semi-jointure : envoyer seulement la clé de jointure
4. Co-localisation : placer les fragments joints sur le même site

Ex. R(10 000 tuples, 100 octets/tuple) → 1 MB à transférer
Après σ (10% sélectivité) → 100 ko seulement
Après π (5 colonnes sur 20) → 25 ko

ÉLIMINATION DE JOINTURES REDONDANTES

Si ∀ tuple ∈ R, ∃ clé correspondante ∈ S (FK garantie) *et* S n'apporte aucune colonne → jointure superflue, supprimable

⚠ **Piège :** Ne supprimer que si la contrainte FK est vérifiée en base — sinon résultat faux

SEMI-JOINTURE DISTRIBUÉE

Définition : `SJ(R, S, A) = R ⋈ π_A(S)`
Envoyer seulement les valeurs de l'attribut de jointure, pas la table entière

Protocole 3 étapes :

1. Site S envoie **`π_A(S)`** → site R (petite projection, ~1 colonne)
2. Site R calcule **`R' = R ⋈ π_A(S)`** → filtre local R
3. Site R envoie **`R'`** → site S pour la jointure finale complète

Facteur de réduction : `fr = |R'| / |R|`
SJ bénéfique si fr << 1 (beaucoup de tuples éliminés)
SJ inutile si fr ≈ 1 (peu de tuples éliminés, 2 transferts pour rien)

Coût SJ : |π_A(S)| + |R'| transferts
vs jointure directe : |R| + résultat
Bénéfice = |R| − (|π_A(S)| + |R'|) = |R| × (1−fr) − |π_A(S)|

Ex. R=1000, S=200, fr=0.08 → R'=80
SJ = 200 ids + 80 tuples = 280 transferts
Direct = 1000 tuples → **gain ×3.6**

⚠ **Piège :** SJ coûteux si fr > 0.5 — vérifier le facteur de réduction avant d'appliquer

VÉRIFICATION RECONSTRUCTIBILITÉ

Complétude : ∪ prédicats_fragments = domaine total de R
Disjonction : prédicats mutuellement exclusifs
Reconstruction : ∪ Ri = R (FH) ou ⋈_PK Ri = R (FV)

Ex. R1 (**`Dur≥12`**), R2 (**`Dur<12`**)
Complétude : Dur≥12 ∪ Dur<12 = ℝ ✓
Disjonction : impossible d'avoir Dur≥12 ET Dur<12 ✓

EXERCICE TYPE — RÉÉCRITURE & RECONSTRUCTION

Données : Table **`R`** fragmentée en :
`R1 = σ_{Dur < 12}(R)`
`R2 = σ_{Dur ≥ 12}(R)`

Requête originale :

```
SELECT * FROM R WHERE Dur=30 OR Dur=6
```

Étape 1 — Réécrire sur fragments :

```
SELECT * FROM R1 WHERE Dur=30 OR Dur=6
UNION
SELECT * FROM R2 WHERE Dur=30 OR Dur=6
```

Étape 2 — Éliminer contradictions :

Ex. R1 (**`Dur<12`**) : Dur=30 impossible (30≥12) → garder seulement **`Dur=6`**
R2 (**`Dur≥12`**) : Dur=6 impossible (6<12) → garder seulement **`Dur=30`**

Résultat optimisé :

```
SELECT * FROM R1 WHERE Dur=6
UNION
SELECT * FROM R2 WHERE Dur=30
```

Vérification reconstructibilité :

Ex. R1 ∪ R2 = R ? Prédicats Dur<12 ∪ Dur≥12 = tous les Dur ✓
Disjonction ? Dur ne peut pas être <12 ET ≥12 simultanément ✓

DEUX APPROCHES D'OPTIMISATION

Rule-Based Optimization (RBO) : l'optimiseur applique un ensemble de règles de réécriture algébrique *sans tenir compte des statistiques* sur les données. L'ordre des règles est fixe et déterministe.

Exemples de règles RBO (RBA) :

- **R1 — Push-down σ :** descendre les sélections au plus près des feuilles de l'arbre algébrique
- **R2 — Push-down π :** projeter le plus tôt possible pour réduire la largeur des tuples
- **R3 — Éliminer σ impossible :** si le prédicat contredit le fragment → retirer
- **R4 — Simplifier jointures :** éliminer jointures redondantes (FK garantie)
- **R5 — Réordonner jointures :** mettre les jointures les plus sélectives en premier

Cost-Based Optimization (CBO) : l'optimiseur génère plusieurs plans candidats, *estime le coût de chacun* (I/O, CPU, réseau) via des statistiques (cardinalités, histogrammes, sélectivités) et retient le plan de moindre coût.

	RBO	CBO
Base	Règles fixes	Statistiques réelles
Qualité	Bonne par défaut	Meilleure si stats à jour
Coût calcul	O(n règles)	Exponentiel (plans)
Risque	Sous-optimal si données atypiques	Mauvais si stats obsolètes

⚠ **Piège :** En pratique : les SGBD modernes (Oracle, PostgreSQL) utilisent **CBO + RBO ensemble** — les règles algébriques réduisent l'espace de plans, puis CBO choisit parmi eux.

HEURISTIQUES — DÉFINITION & EXEMPLES

Heuristique : algorithme qui trouve une solution *bonne mais pas nécessairement optimale*, en temps polynomial, en évitant l'exploration exhaustive de l'espace de recherche.

Pourquoi ? Le problème exact est souvent NP-difficile (ex : ordre optimal des jointures sur n tables = (2n−2)!/(n−1)! plans). L'heuristique sacrifie l'optimalité pour la praticabilité.

Exemples d'heuristiques en BD :

- **Greedy (glouton) :** à chaque étape, choisir localement la meilleure action. Ex : sélectionner la coupure de fragmentation qui réduit le plus le coût immédiat.
- **Nearest neighbor :** ordonner les jointures en commençant par la plus sélective, puis ajouter la table la plus proche (en coût) à chaque pas.
- **Dynamic programming (DP) :** mémoriser les sous-plans optimaux déjà calculés → évite la recalcul. Optimal mais coût $O(3^n)$ → limité à ~10-15 tables.
- **Simulated annealing / Genetic :** pour >15 tables, explorer l'espace des plans de façon stochastique.

Ex. Heuristique glouton (ex1) : initialiser F={R}, puis répéter : évaluer toutes les coupures (σ ou π) sur chaque fragment, appliquer celle qui réduit le plus le coût total, jusqu'à |F|=K.
→ Sous-optimal mais correct et $O(K^2 \times m \times (n_pred + n_attr))$

⚠ **Piège :** Une heuristique ne garantit *pas* l'optimum global — elle peut rater la vraie solution si le gain local mène à un mauvais optimum local.

PROBLÈME D'OPTIMISATION À CONTRAINTE

Forme générale :
Minimiser (ou maximiser) une *fonction objectif* f(x)
sous des *contraintes* : g(x) ≤ K, h(x) = 0, …
→ trouver x* tel que f(x*) est minimal parmi les x valides.

Exemple : fragmentation mixte (ex1, Q2)

Variables : choix des prédicats horizontaux P_H et projections verticales P_V définissant la fragmentation F

Fonction objectif :
`Min Σ freq(q_i) × coût(q_i, F)`
(coût total d'exécution de la charge de requêtes)

Contrainte principale :
`|F| ≤ K_max`
(nombre maximal de fragments admissible)

Contraintes de correction (obligatoires) :
• Complétude : tout tuple ∈ R est dans ≥ 1 fragment
• Reconstruction : ∪⋈ F_i = R
• Disjonction (FH) : chaque tuple dans 1 seul fragment H

⚠ **Piège :** Omettre les contraintes de correction produit une solution qui minimise le coût mais ne permet plus de reconstituer R — la fragmentation est inutilisable

Complexité du problème exact :

Ex. L'espace de recherche est de taille $O(K^2 \times (n_pred + 2^{n_attr}))$
→ exponentiel en n_attr à cause des 2^n_attr projections verticales possibles
→ Variante polynomiale : restreindre aux groupements d'attributs prédéfinis → $O(K^2 \times m \times (n_pred + n_groupes))$

Lien RBO/CBO/Heuristique :
RBO = contraintes de correction appliquées comme règles fixes (pas de coût)
CBO = minimisation explicite de f(x) = coût estimé
Heuristique = résoudre le problème CBO de façon approchée en temps polynomial

BNLJ — MÉCANISME & FORMULE

Principe : charger un lot de M blocs de R en mémoire, parcourir S entièrement pour trouver les correspondances, puis passer au lot suivant de R.

```
Pour chaque lot de M blocs de R :
  Charger le lot en mémoire
  Pour chaque bloc de S :
    Pour chaque tuple r du lot :
      Pour chaque tuple s du bloc :
        Si r.A = s.B → émettre (r, s)
```

Coût = B(R) + ⌈B(R) / M⌉ × B(S)

Variable	Signification
`B(R)`	Blocs table *externe* — lue 1 fois en tout
`B(S)`	Blocs table *interne* — relue intégralement à chaque lot
`M`	Buffer en blocs (1 slot réservé pour lire S bloc à bloc)
`⌈B(R)/M⌉`	Nombre de lots = nombre de fois où S est relue

Mnémo : S paie le prix de chaque lot de R — plus M est grand, moins il y a de lots, moins S est relue.

⚠ **Piège :** Toujours mettre la *plus petite* table en interne (S) pour minimiser ⌈B(R)/M⌉ × B(S)

CAS LIMITES IMPORTANTS

M = 1 : 1 tuple de R à la fois → ⌈B(R)/1⌉ = B(R) lots → Coût = B(R) + B(R)×B(S) ≈ B(R)×B(S) — catastrophique

S tient en mémoire (B(S) ≤ M−1) : charger S une seule fois → 1 seul lot sur R → Coût = B(R) + B(S) — optimal

Mémoire optimale : M = √B(R) + 1 → ⌈B(R)/M⌉ ≈ √B(R) → Coût ≈ 3·B(R) (si B(R) ≤ B(S))

Ex. B(R)=900 → M_opt = √900+1 = 31 → ⌈900/31⌉=29 lots → Coût = 900 + 29×B(S)

SCÉNARIOS NUMÉRIQUES COMPARÉS

B(Livraison)=2000, B(Fournisseur)=2000, débit=10 000 blocs/s

Scénario	Lots	Coût (blocs)	Durée
M=1, ext=Liv.	2000	2000 + 2000×2000 = **4 002 000**	~6m40
M=20, ext=Liv.	100	2000 + 100×2000 = **202 000**	~20s
M=20 + σ (Fournisseur → 40 blocs)	100	2000 + 100×40 = **6 000**	~0.6s
Fournisseur filtré (40 blocs) tient en M=20+	1	2000 + 40 = **2 040**	~0.2s
Inversion : Fournisseur ext., Liv. int.	100	2000 + 100×2000 = **202 000**	~20s

Leçon : le push-down σ peut diviser le coût par ×100 — toujours filtrer avant de joindre

DÉRIVER M MINIMAL & M OPTIMAL

M minimal pour coût cible C :
`B(R) + ⌈B(R)/M⌉ × B(S) ≤ C`
`⌈B(R)/M⌉ ≤ (C − B(R)) / B(S)`
`M ≥ B(R) / floor((C − B(R)) / B(S))`

Ex. B(R)=1000, B(S)=500, C=6000
⌈1000/M⌉ ≤ (6000−1000)/500 = 10
M ≥ 1000/10 = **100 blocs**

M optimal (coût minimum) :
`M_opt = ⌊√B(R)⌋ + 1`
Coût obtenu ≈ B(R) + √B(R) × B(S)
Si B(R) = B(S) → Coût ≈ 3·B(R)

Ex. B(R)=400, B(S)=800
M_opt = √400+1 = 21
Coût = 400 + ⌈400/21⌉×800 = 400 + 20×800 = **16 400 blocs**

⚠ **Piège :** M_opt minimise les I/O mais pas nécessairement le temps si B(R) ≠ B(S) — vérifier aussi l'inversion des rôles

EXERCICE TYPE — CALCULER & COMPARER

Données : R : 500 blocs, **S** : 300 blocs, M = 15 blocs, débit = 5 000 blocs/s

Q1 — BNLJ(R ext., S int.) :

Ex. ⌈500/15⌉ = 34 lots
Coût = 500 + 34×300 = **10 700 blocs** (~2.1s)

Q2 — BNLJ(S ext., R int.) :

Ex. ⌈300/15⌉ = 20 lots
Coût = 300 + 20×500 = **10 300 blocs** (~2.1s)
→ Mettre la plus petite en externe est légèrement meilleur

Q3 — σ réduit S à 30 blocs, BNLJ(R ext., S_filtré int.) :

Ex. S_filtré (30 blocs) < M (15)? Non, 30 > 15
⌈500/15⌉ = 34 lots → 500 + 34×30 = **1 520 blocs**
→ ×7 moins cher qu'avant le filtre

Q4 — σ réduit S à 10 blocs (S tient en M) :

Ex. B(S_filtré)=10 ≤ M−1=14 → 1 seul lot
Coût = 500 + 10 = **510 blocs** (~0.1s)
→ Cas optimal : S chargée une seule fois

Q5 — M minimal pour coût ≤ 2 000 blocs (sans filtre) :

Ex. 500 + ⌈500/M⌉×300 ≤ 2000
⌈500/M⌉ ≤ 5 → M ≥ 500/5 = **100 blocs**

Q6 — M optimal :

Ex. M_opt = √500+1 ≈ 24
Coût = 500 + ⌈500/24⌉×300 = 500 + 21×300 = **6 800 blocs**

HASH JOIN 2 PHASES — MÉCANISME

Phase 1 — Partition : appliquer h(attribut_join) sur R et S → k partitions. Tuples avec même valeur → même partition. Écrire toutes les partitions sur disque.

Phase 2 — Probe : pour chaque paire (Ri, Si), charger Ri en RAM (table de hachage), parcourir Si et sonder.

```
Phase 1 : lire R (1×) + écrire R (1×) = 2·B(R)
          lire S (1×) + écrire S (1×) = 2·B(S)
Phase 2 : lire R (1×) + lire S (1×)  = B(R)+B(S)
---------------------------------------------------
Total   = 3·(B(R) + B(S))
```

Contrainte M : chaque partition Ri doit tenir en mémoire → M ≥ √B(R) + 1

⚠ **Piège :** Si M < √B(R) → partitions trop grandes → spill → Hash Join récursif → coût > 3·(B(R)+B(S))

Ex. B(R)=400, B(S)=200, M=25
M ≥ √400+1 = 21 ✓ → Coût = 3×(400+200) = **1 800 blocs**

SORT-MERGE JOIN — MÉCANISME

Phase 1 : tri externe de R et S sur l'attribut de jointure (2 passes si B(R) > M²)

Phase 2 : fusion des deux listes triées en 1 passage linéaire — 2 pointeurs avancent en parallèle

Coût : tri = 2·B(R) + 2·B(S) (écrire+relire) + merge = B(R)+B(S) → **3·(B(R)+B(S))** au total

Si données déjà triées (index ou ORDER BY préalable) : seulement la phase de merge → Coût = **B(R)+B(S)**

⚠ **Piège :** Sort-Merge supporte les jointures non-équi (**<**, **>**) — Hash Join ne le peut pas

COMPARAISON DES 4 ALGORITHMES

Algo	Coût	Favorable si…	Interdit si…
BNLJ	B(R)+⌈B(R)/M⌉ ×B(S)	S petite ou tient en RAM	M=1, grandes tables
Hash Join	3(B(R)+B(S))	Équijoin, non triées	M < √B(R), non-équi
Sort-Merge	3B(R)+3B(S) ou B(R)+B(S)	Déjà triées, non-équi	Sans index, coûteux
Index NLJ	B(R) + |R|×coût_idx	Index clustered sur S, S petite	Non-clustered + gros résultat

Quand BNLJ bat Hash Join : si B(S) ≤ M−1 (S tient en RAM) → BNLJ coût = B(R)+B(S) < 3(B(R)+B(S))

Quand Index NLJ bat tout : S très sélective (peu de tuples qualifiants) + index clustered → accès quasi-directs, moins de blocs que 3(B(R)+B(S))

LECTURE D'UN PLAN ORACLE (EXPLAIN)

Se lit **de bas en haut** : les feuilles s'exécutent en premier, chaque nœud parent reçoit le résultat de ses enfants.

```
HASH JOIN                                  ← jointure finale
  HASH GROUP BY                            ← agrégation
    TABLE ACCESS FULL : Commandes          ← scan (pas d'index)
  INDEX RANGE SCAN : idx_clients_id        ← B-Tree sur id
    TABLE ACCESS BY ROWID : Clients        ← accès physique
```

Nœud Oracle	Signification
`TABLE ACCESS FULL`	Scan complet — index ignoré ou absent
`INDEX RANGE SCAN`	B-Tree, prédicat range ou égalité avec préfixe
`INDEX UNIQUE SCAN`	B-Tree, égalité sur clé unique → 1 entrée
`BITMAP AND/OR`	Intersection/union de bitmaps (OLAP)
`NESTED LOOPS`	BNLJ ou Index NLJ
`HASH JOIN`	Hash Join 2 phases
`MERGE JOIN`	Sort-Merge Join
`SORT ORDER BY`	Tri explicite (ORDER BY sans index trié)

EXERCICE TYPE — COMPARER & LIRE UN PLAN

Données : `Commandes` B=800, `Clients` B=50, M=30
Index B-Tree clustered sur `Clients.id`. Sélectivité σ_{pays='FR'} = 20% sur Clients.

Q1 — BNLJ (Commandes ext.) :

Ex. ⌈800/30⌉=27 lots → 800 + 27×50 = **2 150 blocs**

Q2 — Hash Join :

Ex. M ≥ √50+1 = 8.1 → M=30 ✓
Coût = 3×(800+50) = **2 550 blocs**

Q3 — BNLJ après push-down σ_{pays='FR'} sur Clients :

Ex. B(Clients_filtré) = 50×0.2 = 10 blocs
10 ≤ M−1=29 → Clients tient en RAM → 1 seul lot
Coût = 800 + 10 = **810 blocs** ✓ meilleur plan

Q4 — Quel plan Oracle correspond au Q3 ?

```
NESTED LOOPS
  TABLE ACCESS FULL : Commandes
  INDEX RANGE SCAN : idx_clients_id
    FILTER: pays='FR'
```

Ex. NESTED LOOPS car Clients filtrés tiennent en RAM → BNLJ optimal
INDEX RANGE SCAN → B-Tree sur id pour la jointure
FILTER push-down avant l'accès physique

Q5 — Sort-Merge si Commandes déjà triée sur client_id :

Ex. Phase tri Commandes = 0 (déjà triée), tri Clients = 3×50=150
Merge = 800+50 = 850 → Total = **1 000 blocs**

ÉNONCÉ — FOURNISSEUR / LIVRAISON

Schéma :

```
Fournisseur(NF, NomF, Adresse)
Livraison(NP, NF, Date, Quantité)
```

Paramètres :

- 10 000 tuples chacune · facteur de blocage : 5 tuples/bloc
- ≈ 100 tuples de Livraison concernent le produit P1
- E/S disque : 20 ms/bloc · mémoire : 100 tuples max

Requête Q : lister les noms des fournisseurs qui livrent P1.

Convertir en blocs (étape préliminaire) :

B(F) = 10 000 / 5 = **2 000 blocs**
B(L) = 10 000 / 5 = **2 000 blocs**
B(L1) = 100 / 5 = **20 blocs** (L filtré sur NP='P1')
Buffer = 100 tuples = **20 blocs**

Questions :

1. Écrire la requête en algèbre relationnelle et SQL
2. Calculer le coût du **Plan 1** : BNLJ naïf, 1 bloc de F à la fois, sélection après jointure
3. Calculer le coût du **Plan 2** : BNLJ, buffer 100 tuples (20 blocs)
4. Calculer le coût du **Plan 3** : filtrer L → L1 d'abord, puis BNLJ(F ext., L1 int.)
5. Calculer le coût du **Plan 4** : filtrer L → L1, puis BNLJ(L1 ext., F int.)
6. Convertir chaque coût en temps (ms). Quel est le plan optimal et pourquoi ?

⚠ **Piège :** Piège : raisonner en tuples au lieu de blocs. Le coût I/O se mesure en lectures de blocs.

⚠ **Piège :** Piège : croire que Plan 3 est optimal. L'inversion des rôles (Plan 4) peut encore améliorer.

Résolution complète

1. Algèbre & SQL :

```
π_NomF( σ_{NP='P1'}(Livraison) ⋈_NF Fournisseur )

SELECT F.NomF
FROM Fournisseur F JOIN Livraison L ON F.NF = L.NF
WHERE L.NP = 'P1';
```

Ex. Sélection *avant* la jointure en algèbre = push-down σ = optimisation clé

2. Plan 1 — BNLJ naïf (lot = 1 bloc, sélection après) :
F externe (2000 blocs), L interne, 1 bloc de F à la fois → 2000 lots

Ex. Lectures F = 2 000
Lectures L = 2 000 × 2 000 = 4 000 000
Total = 4 002 000 blocs → 4 002 000 × 20ms = **~22h13**

3. Plan 2 — BNLJ buffer 20 blocs :
F externe, buffer = 20 blocs → ⌈2000/20⌉ = 100 lots

Ex. Lectures F = 2 000
Lectures L = 100 × 2 000 = 200 000
Total = 202 000 blocs → × 20ms = **~1h07**
Gain vs Plan 1 : ÷ 20

4. Plan 3 — Push-down σ puis BNLJ(F ext., L1 int.) :
Phase 1 : scanner L pour construire L1 (100 tuples, 20 blocs)
Phase 2 : BNLJ F externe (100 lots), L1 interne (20 blocs)

Ex. Phase 1 = 2 000 blocs (scan L)
Lectures F = 2 000 · Lots = 100
Lectures L1 = 100 × 20 = 2 000
Total = 6 000 blocs → × 20ms = **2 minutes**
Gain vs Plan 1 : ÷ 667

5. Plan 4 — Push-down σ puis BNLJ(L1 ext., F int.) :
L1 = 20 blocs ≤ buffer 20 blocs → L1 tient en 1 seul lot → F parcouru 1 fois

Ex. Phase 1 = 2 000 blocs (scan L)
Lots L1 = ⌈20/20⌉ = **1 lot**
Lectures F = 1 × 2 000 = 2 000
Total = 4 000 blocs → × 20ms = **80 secondes**
Gain vs Plan 3 : ÷ 1,5

6. Synthèse & plan optimal :

Plan	Principe	Blocs	Temps
1	BNLJ naïf, lot=1	4 002 000	~22h
2	BNLJ buffer 20 blocs	202 000	~1h07
3	Push-down + BNLJ(F,L1)	6 000	2 min
4	Push-down + BNLJ(L1,F) ★	4 000	80 s

Plan 4 est optimal car : (1) push-down réduit L à 20 blocs, (2) L1 tient dans le buffer → 1 seul lot → F parcouru une seule fois. La clé : *mettre la table la plus petite en externe quand elle tient en mémoire*.

STRUCTURE INTERNE D'UN B-TREE

Arbre équilibré : toutes les feuilles sont à la même profondeur. Hauteur typique = 3-4 niveaux pour des millions de tuples.

```
        [50 | 80]            ← racine (nœud interne)
       /    |    \
[20|35] [60|70] [85|95]   ← nœuds internes
   ↓↓↓    ↓↓↓     ↓↓↓
(val→RID) …                ← feuilles (entrées triées)
```

Nœuds internes : contiennent des clés de routage. Servent uniquement à guider la descente.
Feuilles : contiennent les couples **`(valeur_clé, RID)`** où RID = pointeur vers le tuple physique. Les feuilles sont chaînées (scan en plage).

Recherche : descendre depuis la racine en comparant les clés → O(log n). Pour 1M tuples ≈ 20 entrées/nœud → hauteur ≈ 4 niveaux.

Scan en plage : atteindre la première feuille, puis parcourir la chaîne de feuilles vers la droite jusqu'à la limite haute.

CLUSTERED VS NON-CLUSTERED

	Clustered	Non-Clustered
Tuples physiques	Triés selon l'index	Non triés (ordre quelconque)
Nombre par table	1 seul possible	Plusieurs possibles
Accès plage	Très efficace (séquentiel)	Aléatoire (1 I/O par tuple)
Coût INSERT/UPDATE	Plus élevé (réorganisation)	Plus faible

Ex. Clustered sur **`date`** → **`WHERE date BETWEEN '2024-01' AND '2024-06'`** lit des blocs contigus. Non-clustered : chaque tuple qualifiant peut être dans un bloc différent.

⚠ **Piège :** Avec un index non-clustered qui ramène beaucoup de tuples, chaque tuple = 1 accès disque aléatoire → peut être plus lent qu'un scan complet

RÈGLE DU PRÉFIXE (INDEX COMPOSITE)

Un index composite **`(A, B, C)`** impose que **A doit être présent dans le prédicat** pour que l'index soit utilisable. Sans A, l'arbre ne peut pas être traversé dans le bon ordre.

Prédicat WHERE	idx(A,B,C) ?	Pourquoi
`A = 1`	✓ Oui	Préfixe A présent
`A = 1 AND B > 5`	✓ Oui	Préfixe A + filtre B
`A = 1 AND B = 3 AND C = 2`	✓ Oui	Toutes les colonnes
`B = 3`	✗ Non	A absent → ordre cassé
`B = 3 AND C = 2`	✗ Non	A absent
`A = 1 AND C = 2`	~ Partiel	A utilisé, C filtré à la volée

Ex. **`idx(dept, annee, salaire)`**
✓ **`WHERE dept='RH'`**
✓ **`WHERE dept='RH' AND annee=2022`**
✗ **`WHERE annee=2022`** — dept absent

COVERING INDEX (INDEX DE COUVERTURE)

Un index est dit *couvrant* si toutes les colonnes du SELECT et du WHERE sont dans l'index → **aucun accès à la table physique** (index-only scan)

Ex. **`CREATE INDEX cov ON Emp(dept, annee, salaire); SELECT salaire FROM Emp WHERE dept='RH' AND annee=2022`**
→ Toutes les colonnes (dept, annee, salaire) dans l'index → **0 accès table**

Sélectivité : si la requête ramène > 5-10% des tuples, l'optimiseur préfère un scan complet (moins d'I/O aléatoires qu'un index non-clustered)

⚠ **Piège :** Trop d'index = maintien coûteux sur écritures. Chaque INSERT/UPDATE doit mettre à jour tous les index de la table.

EXERCICE TYPE — PRÉFIXE & COÛT

Table : **`Commandes(id, client_id, region, statut, montant, date_cmd)`**
500 000 tuples · 2 000 blocs
Index : **`idx1(region, statut, montant)`** (non-clustered)
Cardinalités : region=8, statut=4, montant élevée

Q1 — `WHERE region='Sud' AND statut='LIVRE'`

Ex. Préfixe region ✓ → idx1 utilisé
Sélectivité : 1/8 × 1/4 = 3.1% → sous le seuil 5-10%
→ **Index range scan** sur idx1

Q2 — `WHERE statut='LIVRE'`

Ex. region absent → préfixe non respecté
→ idx1 **inutilisable** → scan complet (2 000 blocs)

Q3 — `WHERE region='Sud' ORDER BY montant DESC`

Ex. region ✓ → idx1 peut filtrer region. Mais ORDER BY montant : montant est en 3ème position et statut n'est pas filtré → tri dans l'index cassé
→ **Index range scan sur region + tri externe** sur montant

Q4 — Covering index pour `SELECT montant FROM … WHERE region='Sud' AND statut='LIVRE'`

Ex. idx1(region, statut, montant) : region ✓, statut ✓, montant dans SELECT
→ **Index-only scan** — 0 accès table physique ✓

Q5 — Si 40% des tuples sont region='Sud' ?

Ex. 40% >> seuil 10% → optimiseur **ignore l'index** et préfère scan complet (2 000 blocs séquentiel vs ~800 000 I/O aléatoires)

INDEX BITMAP — FONCTIONNEMENT

Pour chaque valeur distincte d'un attribut, on stocke un **vecteur de N bits** (1 bit par tuple de la table) : bit=1 si le tuple a cette valeur.

```
Table (6 tuples) :
id  couleur  taille
1   ROUGE    L
2   BLEU     M
3   VERT     S
4   ROUGE    L
5   BLEU     L
6   ROUGE    M

Bitmap couleur :        Bitmap taille :
ROUGE : 1 0 0 1 0 1    L : 1 0 0 1 1 0
BLEU  : 0 1 0 0 1 0    M : 0 1 0 0 0 1
VERT  : 0 0 1 0 0 0    S : 0 0 1 0 0 0
```

Requête multi-critères : `WHERE couleur='ROUGE' AND taille='L'`
→ AND bit-à-bit : `101001 AND 100110 = 100000`
→ Résultat : tuple 1 seulement (bit 1 à position 1)

OR : `WHERE couleur='ROUGE' OR couleur='BLEU'`
→ OR bit-à-bit : `100101 OR 010010 = 110111` → tuples 1,2,4,5,6

BITMAP — QUAND ET POURQUOI

	Bitmap	B-Tree
Cardinalité	Faible (≤100 val.)	Haute (milliers+)
Requêtes	Multi-AND/OR analytiques	Égalité, plage, tri
Contexte	OLAP (lecture)	OLTP (lecture+écriture)
Espace	N bits / valeur distincte	Proportionnel à N tuples

⚠ **Piège :** Bitmap très coûteux en écriture concurrente : verrouillage au niveau du vecteur entier → **à éviter en OLTP** (INSERT/UPDATE fréquents)

⚠ **Piège :** Haute cardinalité → vecteur sparse (99% de zéros) → peu d'avantage, gaspillage mémoire

Ex. `region` (8 valeurs) → Bitmap idéal. `email` (unique) → Bitmap catastrophique.

BJI — POURQUOI LE B-TREE NE SUFFIT PAS

Dans un schéma étoile, la table Faits est énorme. Filtrer sur plusieurs dimensions simultanément avec des B-Trees ordinaires oblige à :
1. Lire tous les tuples Faits liés à chaque dimension filtrée
2. Faire l'intersection en mémoire → toujours beaucoup d'I/O sur Faits

BJI solution : pour chaque paire (Dimension D, Table Faits F), stocker directement `D_RID → liste de F_RIDs`. La jointure est *pré-calculée*.

```
-- Exemple BJI(Client, Ventes) :
Client RID 42 (pays='France')  → Ventes RIDs
{1,5,8,12,99}
Client RID 43 (pays='Germany') → Ventes RIDs {3,7,11}
Client RID 44 (pays='France')  → Ventes RIDs {2,6,17,88}
```

BJI — PIPELINE D'EXÉCUTION DÉTAILLÉ

Requête : `SUM(montant) WHERE pays='France' AND cat='Electronique'`

1. **Filtre dim. Client :** σ_{pays='France'}(Client) → D_RIDs = {42, 44}
2. **Lookup BJI_Client :** RIDs 42 et 44 → F_RIDs_A = {1,2,5,6,8,12,17,88,99}
3. **Filtre dim. Produit :** σ_{cat='Electro'}(Produit) → D_RIDs = {10, 11}
4. **Lookup BJI_Produit :** → F_RIDs_B = {1,4,6,12,30,88}
5. **Intersection :** F_RIDs_A ∩ F_RIDs_B = {1, 6, 12, 88}
6. **Accès physique Faits :** lire *seulement* les 4 tuples {1,6,12,88}

Clé : étapes 1-5 s'exécutent sur des entiers en mémoire — **0 accès physique à Faits** avant l'étape 6

BJIs activés	Réduction accès Faits
1 BJI (1 filtre dim.)	~−41%
2 BJIs (2 filtres dim.)	~−70%
3 BJIs (3 filtres dim.)	~−93%

⚠ **Piège :** BJI coûteux à maintenir (INSERT dans Faits = mise à jour de tous les BJIs concernés) → pertinent uniquement en **lecture intensive (OLAP)**

INDEX DE HACHAGE (HASH INDEX)

Principe : `h(valeur)` transforme la clé en numéro de *bucket*. Chaque bucket contient les RIDs des tuples ayant cette valeur → accès direct O(1).

```
h("Paris") = 3 → bucket 3 → {RID 12, RID 47}
h("Lyon")  = 7 → bucket 7 → {RID 5, RID 91}
Recherche "Paris" : 1 calcul → bucket 3 → résultat
```

Opération	Hash	B-Tree
Égalité **=**	✓ O(1)	✓ O(log n)
Plage **BETWEEN**	✗	✓
ORDER BY	✗	✓
LIKE 'a%'	✗	✓

Pourquoi plages impossibles : `h("Paris")=3` et `h("Lyon")=7` n'ont aucun ordre — valeurs proches dispersées dans des buckets arbitraires. Impossible de "parcourir" l'index de façon ordonnée.

⚠ **Piège :** B-Tree presque toujours préféré car il couvre égalité *et* plages. PostgreSQL propose Hash Index mais l'optimiseur CBO choisit souvent B-Tree.

Hash Join ≠ Hash Index : Hash Join (p.4) hache les tuples *en mémoire temporairement* pendant la jointure — pas un index persistant sur disque.

EXERCICE TYPE — CHOISIR & CALCULER

Schéma : `Emp(id, dept, job, salaire, annee)` — 500 000 tuples
Cardinalités : dept=20, job=50, annee=30, salaire élevée

Q1 — `WHERE dept='RH' AND job='DBA'`

Ex. dept (20 val.) + job (50 val.) → faible card. → **Bitmap(dept) AND Bitmap(job)**
Opération : AND bit-à-bit → 1 passage sur 500 000 bits ≈ 60 Ko → très rapide

Q2 — `WHERE salaire BETWEEN 40k AND 60k ORDER BY salaire`

Ex. salaire haute card. + ORDER BY → **B-Tree clustered sur salaire**
Bitmap impossible (trop de valeurs distinctes)

Q3 — `WHERE dept='RH' AND annee>=2020`

Ex. dept (Bitmap ✓) + annee (30 val., Bitmap ✓)
→ **Bitmap(dept) AND Bitmap(annee≥2020)**
Pour annee≥2020 : OR des bitmaps 2020, 2021, 2022, 2023, 2024 puis AND avec dept

Q4 — Schéma étoile, Faits(10M) + dim. Client(pays) + dim. Produit(cat)

Ex. Sans BJI : filtre pays → ~1M tuples Faits lus, puis filtre cat → ~100k tuples
Avec 2 BJIs : intersection RIDs en mémoire → accès direct aux ~70k tuples qualifiants
→ **Réduction ~93% des I/O sur Faits**

Règle de sélection :
card ≤ 100 → Bitmap

OLTP VS OLAP

	OLTP	OLAP
But	Transactions unitaires	Analyses agrégées
Requêtes	INSERT/UPDATE/DELETE rapides	SELECT + GROUP BY complexes
Volumétrie	Quelques lignes par requête	Millions de lignes scannées
Modèle	Normalisé (3NF)	Dénormalisé (étoile/flocon)
Index	B-Tree sur PK/FK	Bitmap, BJI, VM
Fraîcheur	Temps réel	Tolérance au délai (batch)

OPÉRATIONS OLAP — AVEC SQL

- **Roll-up :** agréger vers le haut (jour → mois → année)
  `GROUP BY annee` au lieu de **`GROUP BY jour`**
- **Drill-down :** décomposer vers le bas (pays → région → ville)
  Ajouter **`region`** au GROUP BY
- **Slice :** fixer une valeur d'une dimension
  `WHERE annee = 2024`
- **Dice :** sous-cube multi-dimensions
  `WHERE annee IN (2023,2024) AND region='Nord'`
- **Pivot :** croiser deux dimensions en lignes/colonnes
  `CASE WHEN region='Nord' THEN montant END`

ROLLUP SQL : génère automatiquement les sous-totaux
`GROUP BY ROLLUP(pays, region, ville)`
→ produit : (pays,region,ville) + (pays,region) + (pays) + ()

CUBE SQL : toutes les combinaisons possibles
`GROUP BY CUBE(pays, annee)`
→ produit : (pays,annee) + (pays) + (annee) + ()

SCHÉMA ÉTOILE — STRUCTURE

```
      Client(id, nom, ville, pays)
           ↑ FK_Client
Ventes(FK_Client, FK_Produit, FK_Date,
       montant, quantite, remise)     ← Faits
           ↓ FK_Produit  ↓ FK_Date
     Produit(id, nom,  Date(id, jour,
      cat, prix)        mois, annee,
                        trimestre)
```

Règles table Faits : seulement FK + mesures numériques — jamais de libellés (ceux-ci sont dans les dimensions)

Étoile vs Flocon :
Étoile : dimensions dénormalisées → 1 jointure/dimension
Flocon : dimensions normalisées → plusieurs jointures/dimension, moins de redondance mais plus lent

⚠ **Piège :** Flocon : Produit → Catégorie → Rayon nécessite 2 jointures supplémentaires par requête — souvent évité en OLAP pur

REQUÊTE OLAP TYPE + OPÉRATIONS

```
-- Roll-up : ventes par pays et année
SELECT C.pays, D.annee,
       SUM(V.montant)  AS total,
       COUNT(*)        AS nb_ventes
FROM Ventes V
JOIN Client C ON V.FK_Client = C.id
JOIN Date   D ON V.FK_Date   = D.id
WHERE C.pays IN ('France','Espagne') -- Dice
  AND D.annee BETWEEN 2023 AND 2025  -- Slice
GROUP BY C.pays, D.annee             -- Roll-up
ORDER BY D.annee, total DESC;
```

```
-- Drill-down : ajouter la région
GROUP BY C.pays, C.region, D.annee
```

```
-- ROLLUP automatique : totaux par pays + grand total
GROUP BY ROLLUP(C.pays, D.annee)
```

EXERCICE TYPE — REQUÊTE OLAP + INDEX + OPÉRATIONS

Schéma : Ventes(10M), Client(pays, region), Produit(cat), Date(jour, mois, trimestre, annee)

Q1 — Requête "ventes par catégorie en 2024 pour la région Nord" :

```
SELECT P.cat, SUM(V.montant), COUNT(*)
FROM Ventes V
JOIN Client C  ON V.FK_Client  = C.id
JOIN Produit P ON V.FK_Produit = P.id
JOIN Date D    ON V.FK_Date    = D.id
WHERE C.region = 'Nord' AND D.annee = 2024
GROUP BY P.cat
ORDER BY SUM(V.montant) DESC;
```

Q2 — Index optimaux :

Ex. region (5 val.) → Bitmap · annee (20 val.) → Bitmap
montant → B-Tree (ORDER BY) · 2 BJIs → -70% accès Faits

Q3 — Roll-up vers l'année puis drill-down vers le trimestre :

Ex. Roll-up : **`GROUP BY D.annee`** (résultat agrégé sur 2024 entier)
Drill-down : **`GROUP BY D.annee, D.trimestre`** (décompose en 4 trimestres)

Q4 — Générer automatiquement tous les niveaux (pays, région, total) :

```
GROUP BY ROLLUP(C.pays, C.region)
```

Ex. Résultats : (pays, region) + (pays) + () = lignes détail + sous-totaux pays + grand total

Q5 — Slice : restreindre à 2024, Dice : France + Espagne

Ex. Slice = **`WHERE D.annee=2024`** (1 valeur fixée)
Dice = **`WHERE C.pays IN ('France','Espagne') AND D.annee IN (2023,2024)`**

VUES MATÉRIALISÉES — SYNTAXE

```
CREATE MATERIALIZED VIEW mv_ventes AS
  SELECT region, annee,
         SUM(montant) AS total, COUNT(*) AS nb
  FROM Ventes GROUP BY region, annee
WITH DATA;                      -- calculer à la création

CREATE UNIQUE INDEX ON mv_ventes(region, annee);
-- Obligatoire pour CONCURRENTLY

REFRESH MATERIALIZED VIEW mv_ventes;            --
bloquant
REFRESH MATERIALIZED VIEW mv_ventes CONCURRENTLY; -- non-
bloquant

DROP MATERIALIZED VIEW mv_ventes;  -- supprimer
```

⚠ **Piège :** CONCURRENTLY : lectures non bloquées pendant le refresh, mais nécessite un index unique — sinon erreur PostgreSQL

WITH NO DATA : crée la VM vide (structure seulement), utile pour planifier le premier REFRESH manuellement

STRATÉGIES & CRITÈRES DE CHOIX

Stratégie	Déclencheur	Usage
On-demand	REFRESH manuel	Rapports ponctuels
On-commit	Après chaque commit	Données critiques temps réel
Scheduled	pg_cron, cron	Dashboards périodiques

VM utile si : requête coûteuse + fréquente + données stables
VM inutile si : données très volatiles, requête exécutée 1 seule fois, résultat déjà indexé efficacement

Vue simple vs VM : VIEW = alias, recalculé à chaque appel. VM = résultat *stocké physiquement sur disque*.

ANALYSE COÛT : DIRECT VS VM

Coût direct : freq × coût_requête
Coût VM : freq × coût_lecture_VM + nb_refresh/jour × coût_refresh

Ex. Requête : JOIN 3 tables, 10M lignes → 50ms. Exécutée 1000×/jour.
Coût direct = 1000×50 = **50 000ms**
VM : lecture=0.5ms, refresh 24×/jour=200ms chacun
Coût VM = 1000×0.5 + 24×200 = 500+4800 = **5 300ms**
→ Gain ×9.4

RÉÉCRITURE TRANSPARENTE

L'optimiseur détecte qu'une requête est *couverte* par une VM et la substitue silencieusement, sans modifier le SQL de l'utilisateur.

Conditions pour la réécriture :
1. La VM contient toutes les colonnes du SELECT
2. Le WHERE de la requête est un sous-ensemble de celui de la VM
3. Le GROUP BY est compatible
4. La VM est fraîche (pas expirée)

⚠ **Piège :** Si la VM est périmée (refresh non effectué), l'optimiseur peut ignorer la réécriture et exécuter la requête directement sur les tables de base

INDEX SUR VM + SQL FENÊTRAGE

```
-- Index composite sur VM (double accélération)
CREATE INDEX ON mv_ventes(region, annee);
-- Agrégat précalculé + accès indexé

-- Fonctions de fenêtrage
RANK()       OVER (PARTITION BY dept ORDER BY sal DESC)
DENSE_RANK() -- pas de trous dans la numérotation
ROW_NUMBER() -- numéro unique même à égalité
LAG(val,1)   OVER (ORDER BY date) -- valeur précédente
LEAD(val,1)  OVER (ORDER BY date) -- valeur suivante
SUM(montant) OVER (PARTITION BY region ORDER BY date
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW)
             -- cumul courant
```

EXERCICE TYPE — VM COMPLÈTE

Contexte : requête analytique exécutée 500×/jour sur 5M lignes (coût = 200ms). Données mises à jour toutes les heures.

Q1 — VM intéressante ?

Ex. Coût direct = 500×200 = **100 000ms/jour**
Coût VM = 500×1ms + 24×200ms = 500+4800 = **5 300ms**
→ **VM très utile** (×19 moins cher)

Q2 — Fréquence refresh :

Ex. Données changent 1×/h → refresh 1×/heure suffisant
Si fraîcheur critique → on-commit (coût write augmente)
Si tolérance 1h → scheduled + CONCURRENTLY

Q3 — Syntaxe + réécriture transparente :

```
CREATE MATERIALIZED VIEW mv AS
  SELECT region, annee, SUM(montant) AS total
  FROM Ventes GROUP BY region, annee
WITH DATA;
CREATE UNIQUE INDEX ON mv(region, annee);
-- pg_cron toutes les heures :
REFRESH MATERIALIZED VIEW mv CONCURRENTLY;

-- Requête utilisateur (réécrite silencieusement) :
SELECT region, SUM(montant) FROM Ventes
WHERE annee=2024 GROUP BY region;
-- → optimiseur utilise mv si couverte ✓
```

Q4 — Conditions de réécriture ici :

Ex. mv contient (region, annee, SUM) ✓
WHERE annee=2024 ⊆ mv (pas de filtre dans mv) ✓
GROUP BY region ⊆ mv(region,annee) ✓
→ réécriture applicable

⚠ **Piège :** Si on ajoute WHERE mois='Janvier' → mois absent de mv → réécriture impossible

ÉNONCÉ

On dispose d'une base distribuée avec les tables suivantes :

```
Employes(id, nom, dept, salaire, site)
Projets(id_proj, titre, budget, site_proj)
```

La table **`Employes`** est fragmentée ainsi :

- **`E1 = σ_{site='Paris'}(Employes)`** stocké sur Site A
- **`E2 = σ_{site='Lyon'}(Employes)`** stocké sur Site B
- Puis pour chaque fragment, projection : **`E1v = π_{id,nom,dept}(E1)`**, **`E1s = π_{id,salaire}(E1)`**

La table **`Projets`** est entière sur Site C.

Questions :

1. Quel type de fragmentation pour **`Employes`** ?
2. Vérifier la reconstructibilité et la disjonction
3. Réécrire la requête suivante sur les fragments :
   `SELECT nom, salaire FROM Employes WHERE site='Paris' AND dept='IT'`
4. Optimiser avec push-down et éliminer fragments inutiles
5. Écrire la requête distribuée finale

Résolution pas-à-pas

1. **Type de fragmentation :**
   E1, E2 = sélection sur **`site`** → **FH** (horizontale)
   E1v, E1s = projection sur colonnes → **FV** (verticale)
   → Fragmentation globale : **Mixte HV** (H d'abord, puis V sur chaque fragment H)
2. **Vérifier reconstructibilité et disjonction :**
   Complétude : site='Paris' ∪ site='Lyon' — couvre-t-il tous les tuples ? Non si ∃ site='Bordeaux'. ⚠ **À vérifier selon le domaine.**
   Disjonction : un employé ne peut avoir site='Paris' ET site='Lyon' simultanément ✓
   Reconstruction : **`Employes = (E1v ⋈_{id} E1s) ∪ (E2v ⋈_{id} E2s)`**
3. **Réécriture initiale :**

```
SELECT nom, salaire FROM E1v, E1s
WHERE E1v.id=E1s.id AND site='Paris' AND dept='IT'
UNION
SELECT nom, salaire FROM E2v, E2s
WHERE E2v.id=E2s.id AND site='Lyon' AND dept='IT'
```

4. **Optimiser — push-down et élimination :**
   Fragment E2 a **`site='Lyon'`** → prédicat **`site='Paris'`** est une *contradiction* → **éliminer E2 complètement**
   Push-down **`dept='IT'`** sur E1v : applicable car E1v contient **`dept`** ✓
   Colonnes **`nom`** ∈ E1v, **`salaire`** ∈ E1s → besoin des deux fragments E1v et E1s
5. **Requête distribuée finale :**

```
-- Exécuté sur Site A uniquement :
SELECT E1v.nom, E1s.salaire
FROM E1v JOIN E1s ON E1v.id = E1s.id
WHERE E1v.dept = 'IT';
```

Aucun transfert réseau nécessaire — tout sur Site A ✓

ÉNONCÉ

Considérez les tables :

```
Commandes(id, client_id, produit_id, montant, date)
Clients(id, nom, region, categorie)
```

Table	Tuples	Blocs	Sélectivité σ
Commandes	500 000	B=1 000	region='Nord' → 5%
Clients	50 000	B=100	categorie='VIP' → 10%

Mémoire disponible : M = 40 blocs.

Index B-Tree sur **`Clients.id`** (clustered).

Requête :

```
SELECT C.nom, SUM(O.montant)
FROM Commandes O
JOIN Clients C ON O.client_id = C.id
WHERE C.categorie = 'VIP'
  AND C.region = 'Nord'
GROUP BY C.nom;
```

Questions :

1. Calculer coût BNLJ (Commandes externe, Clients interne)
2. Calculer coût après push-down σ sur Clients
3. Calculer coût Hash Join
4. Quel plan Oracle choisiriez-vous et pourquoi ?
5. Trouver M minimum pour que BNLJ coûte ≤ 1 500 blocs

Résolution pas-à-pas

1. **BNLJ sans optimisation :**

Commandes externe (B=1000), Clients interne (B=100), M=40

⌈1000/40⌉ = 25 lots

Coût = 1 000 + 25 × 100 = **3 500 blocs**

2. **Après push-down σ (Clients filtrés) :**

σ_{categorie='VIP' AND region='Nord'} → 10% × 5% = **0.5%** de Clients

B(Clients_filtrés) = 100 × 0.005 = **~1 bloc**

Clients filtrés tient en M (1 << 40) → charger une seule fois

Coût = B(Commandes) + B(Clients_filtrés) = 1000 + 1 = **1 001 blocs**

3. **Hash Join :**

M ≥ √B(Clients)+1 = √100+1 = 11 → M=40 suffisant ✓

Coût = 3×(1000 + 100) = **3 300 blocs**

4. **Meilleur plan :**

Push-down BNLJ = 1 001 blocs < Hash Join = 3 300 blocs < BNLJ brut = 3 500 blocs

→ **Plan Oracle optimal :**

```
HASH GROUP BY
  NESTED LOOPS
    TABLE ACCESS FULL : Commandes        ← externe
    TABLE ACCESS (by index) : Clients    ← interne
      INDEX RANGE SCAN : idx_clients_id
    FILTER: categorie='VIP' AND region='Nord'
```

En pratique : appliquer filtre Clients AVANT la jointure

5. **M minimum pour BNLJ ≤ 1 500 blocs (avec clients filtrés B=1) :**

1000 + ⌈1000/M⌉ × 1 ≤ 1500

⌈1000/M⌉ ≤ 500 → M ≥ 1000/500 = 2

→ **M = 2 blocs suffisent** avec le push-down !

ÉNONCÉ

Schéma HR simplifié :

```
Emp(id, nom, dept_id, job_id, salaire, annee)
Dept(id, nom_dept, localisation)
Job(id, titre, sal_min, sal_max)
```

Table	Tuples	Cardinalités attributs
Emp	200 000	dept: 20, job: 50, annee: 30
Dept	20	localisation: 5
Job	50	—

Requête analysée :

```
SELECT D.nom_dept, J.titre, COUNT(*), AVG(E.salaire)
FROM Emp E
JOIN Dept D ON E.dept_id = D.id
JOIN Job  J ON E.job_id  = J.id
WHERE D.localisation = 'Paris'
  AND E.annee >= 2020
GROUP BY D.nom_dept, J.titre;
```

Questions :

1. Proposer les index optimaux (type + justification)
2. Appliquer les règles RBA (R1–R8) pour optimiser le plan logique
3. Un VM serait-il utile ici ? Justifier + écrire la syntaxe
4. Choisir entre REFRESH et REFRESH CONCURRENTLY

Résolution pas-à-pas

1. Index optimaux :

Emp.dept_id : FK de jointure → **B-Tree sur dept_id** (haute card. en jointure)
Emp.job_id : idem → **B-Tree sur job_id**
Emp.annee : card=30, range (**`≥2020`**) → **B-Tree sur annee**
Dept.localisation : card=5, filtre OLAP → **Bitmap sur localisation**
Composite possible : **`idx(dept_id, annee)`** si filtre souvent combiné
Si OLAP étoile : BJI(Dept, Emp) + BJI(Job, Emp) pour accès Emp réduit

2. Règles RBA (Réécriture par l'Algèbre) :

R1 — Push-down σ : appliquer **`localisation='Paris'`** sur Dept avant jointure → Dept_filtrée = 4 tuples (1 localisation sur 5)
R2 — Push-down σ sur Emp : **`annee≥2020`** avant jointure → Emp_filtrée = 200k × (6/30) ≈ 40k tuples
R3 — Push-down π : projeter uniquement les colonnes nécessaires avant jointure (id, dept_id, job_id, salaire, annee)
R4 — Éliminer jointure : Dept jointe mais si toutes les FK Emp.dept_id ont un correspondant → jointure non éliminable ici (on a besoin de nom_dept)
R5 — Ordre des jointures : Dept_filtré (4 tuples) × Emp_filtré (40k) < Emp (200k) × Dept_filtré → joindre Dept d'abord

Plan optimisé :
σ(Dept) → π(Dept) → ⋈_Emp_filtré → ⋈_Job → π finale → GROUP BY

3. VM utile ici ?

Critères : requête complexe (3 tables, agrégat), fréquente (dashboard RH quotidien), données stables (Emp mis à jour en batch)
→ **Oui, VM très utile**

```
CREATE MATERIALIZED VIEW mv_rh_stats AS
  SELECT D.nom_dept, J.titre,
         COUNT(*) AS nb_emp,
         AVG(E.salaire) AS sal_moy
  FROM Emp E
  JOIN Dept D ON E.dept_id = D.id
  JOIN Job  J ON E.job_id  = J.id
  WHERE D.localisation = 'Paris'
    AND E.annee >= 2020
  GROUP BY D.nom_dept, J.titre
WITH DATA;
CREATE UNIQUE INDEX ON mv_rh_stats(nom_dept, titre);
```

4. REFRESH vs CONCURRENTLY :

Si dashboard consulté en continu → **REFRESH CONCURRENTLY** (non bloquant, nécessite l'index unique créé ci-dessus)
Si batch nocturne accepté → **REFRESH** simple (plus rapide)

⚠ **Piège :** Sans index unique → CONCURRENTLY impossible → erreur PostgreSQL

Ex. 2 Exercice type — Requêtes réparties & Fragmentation mixte dérivée

Énoncé

On dispose de deux relations :

- **`R(Enum, Name, Age, Rank)`** — Employés
- **`S(Enum, Jnum, Resp, Dur)`** — Projets (**`Dur`** en mois)

Les fragments définis sont :

Fragment	Définition	Type	Site
S1	$\sigma_{Dur<12}(S)$	FH	Site A
S2	$\sigma_{Dur\geq 12}(S)$	FH	Site B
R1	$\pi_{Enum,Name,Age}(R \bowtie S1)$	Dérivée	Site A
R2	$\pi_{Enum,Rank}(R \bowtie S1)$	Dérivée	Site A
R3	$\pi_{Enum,Name,Age}(R \bowtie S2)$	Dérivée	Site B
R4	$\pi_{Enum,Rank}(R \bowtie S2)$	Dérivée	Site B

Hypothèse : Tout employé de R participe à au moins un projet de S (∀ Enum ∈ R, ∃ Enum ∈ S).

Requête Q :

```
SELECT R.Name
FROM R, S
WHERE R.Enum = S.Enum
  AND (S.Dur = 30 OR R.Age = 55)
```

Questions :

1. Reconstruire R et S depuis les fragments
2. S est-il disjoint ? R est-il disjoint ?
3. Réécrire Q sur les fragments (avant optimisation)
4. Optimiser en utilisant l'hypothèse

⚠ Piège : ⚠ Fragmentation dérivée de R

R1, R2, R3, R4 sont des *fragments dérivés* : leur partitionnement dépend de S (via la jointure R⋈S1/S2), pas d'un prédicat direct sur R. Résultat : R peut être **non-disjoint** si un employé participe à des projets dans les deux fragments de S.

Résolution pas-à-pas

1. **Q1 — Reconstruction de S :**
S1 et S2 forment une partition de S par Dur :
`S = S1 ∪ S2` (union simple, pas de jointure nécessaire)

> Vérification : S1 ∩ S2 = ∅ car Dur<12 XOR Dur≥12 → reconstructibilité OK

2. **Q1 — Reconstruction de R :**
R est fragmenté verticalement après jointure avec S1 et S2 :
$\mathbf{R = (R1 \cup R3) \bowtie_{Enum} (R2 \cup R4)}$

Pourquoi la jointure ? R1/R3 contiennent {Enum,Name,Age}, R2/R4 contiennent {Enum,Rank} → il faut les réunir sur Enum pour reconstituer tous les attributs.
Pourquoi sans perte ? Enum est clé primaire de R → la jointure sur Enum est injective.

3. **Q2 — Disjonction :**
S est disjoint ✓ : Dur<12 et Dur≥12 sont mutuellement exclusifs → aucun tuple dans S1∩S2.
*R est **non-disjoint** ✗* : un employé (ex: Enum=E3) peut avoir un projet avec Dur=6 (dans S1) ET un projet avec Dur=24 (dans S2). Il apparaît donc dans R1 ET dans R3 simultanément.

> **⚠ Piège :** Conséquence : une requête sur R sans précaution peut compter E3 deux fois → il faut traiter les doublons (UNION sans ALL, ou DISTINCT)

4. **Q3 — Réécriture sur fragments (avant optimisation) :**
On substitue R et S par leurs fragments. La condition **`Dur=30 OR Age=55`** génère deux branches :
Branche Dur=30 → Dur=30 ≥ 12 → uniquement dans S2 → fragment R3 (qui porte Age,Name)
Branche Age=55 → Age est dans R1 et R3 (fragments de R côté {Enum,Name,Age})

```
-- Branche 1 : Dur=30 (dans S2 uniquement)
SELECT R3.Name FROM R3, S2
WHERE R3.Enum = S2.Enum AND S2.Dur = 30
UNION
-- Branche 2 : Age=55 côté S1 (Dur<12)
SELECT R1.Name FROM R1
WHERE R1.Age = 55
UNION
-- Branche 3 : Age=55 côté S2 (Dur≥12)
SELECT R3.Name FROM R3
WHERE R3.Age = 55
```

> Note : les branches 2 et 3 n'ont pas besoin de joindre S car on cherche juste le nom de l'employé (pas d'attribut de S dans SELECT). La jointure est implicitement satisfaite via la construction de R1/R3.

5. **Q4 — Optimisation avec l'hypothèse :**
Hypothèse : ∀ Enum ∈ R, ∃ Enum ∈ S → tout employé de R a au moins un projet → la jointure R⋈S n'élimine aucun tuple de R.
Dans les branches Age=55, la jointure avec S était implicite mais inutile (on ne sélectionne rien de S) :
Avant : "Employés de R avec Age=55 qui ont au moins un projet dans S" = jointure obligatoire
Après : "Employés de R avec Age=55" → tous ont un projet par hypothèse → **supprimer la jointure**

```
-- Plan optimisé
SELECT R3.Name FROM R3, S2
WHERE R3.Enum = S2.Enum AND S2.Dur = 30
UNION
SELECT R1.Name FROM R1 WHERE R1.Age = 55  -- sans jointure S
UNION
SELECT R3.Name FROM R3 WHERE R3.Age = 55  -- sans jointure S
```

> **Gain :** branches 2 et 3 deviennent de simples scans de R1/R3 filtrés — élimination d'une semi-jointure coûteuse sur chaque site.